syngenio
Aktiengesellschaft

We make IT work.

Integration der Drools Rule Engine in IT-Architekturen

Referent Michael Frandsen, 14. März 2008

syngenio AG

- Unabhängiges IT-Beratungs- und –Servicehaus
  - Niederlassungen in Bonn, Wiesbaden, Stuttgart und München
  - Vertriebsbüro im Norden Deutschlands
- Gegründet 2001, aktuell 100 Mitarbeiter
- Dienstleistungfelder
  - IT-Beratung
  - Realisierung von Lösungen
  - Software-Betreuung
- Branchenschwerpunkte
  - Finanzdienstleister
  - Telekommunikation

Agenda

- Rule Engine
- Drools
- API – Level – Integration
- Integrationsszenarien
- Zusammenfassung

Agenda

- Rule Engine
- Drools
- API – Level – Integration
- Integrationsszenarien
- Zusammenfassung

Rule Engine - Motivation

- Zentrale Sammelstelle für Geschäftsregeln
- Trennung von Logik und Anwendungen
- Regeln sind lesbarer als Code
  - Nicht-Techniker/Business User/Domänenexperten können Regeln schreiben
- Regeln sind flexibler
- Komplexe Probleme lassen sich oft besser in Regeln beschreiben als in Sourcecode

Beispiel Abseits-Regel

- Die Abseits-Regel gilt als erfüllt, wenn
  - sich zwischen dem angreifenden Spieler A und der Torlinie nur noch ein gegnerischer Spieler befindet
  - sich der angreifende Spieler A in der gegnerischen Spielhälfte befindet
  - ein weiterer Spieler der Mannschaft des Spieler A den Ball nach vorne passt
  - und der angreifende Spieler A aktiv ins Spiel eingreift.

Eine neue (alte) Denkweise…

- Regeln sind deklarativ
- Regeln sind eher SQL-ähnlich

SELECT * FROM VISITOR WHERE AGE > 27

Wenn der Besucher älter als 27 Jahre ist, **dann** ist er älter als ich.

- Regeln kann man nicht aufrufen wie Methoden

Regeln sind deklarativ

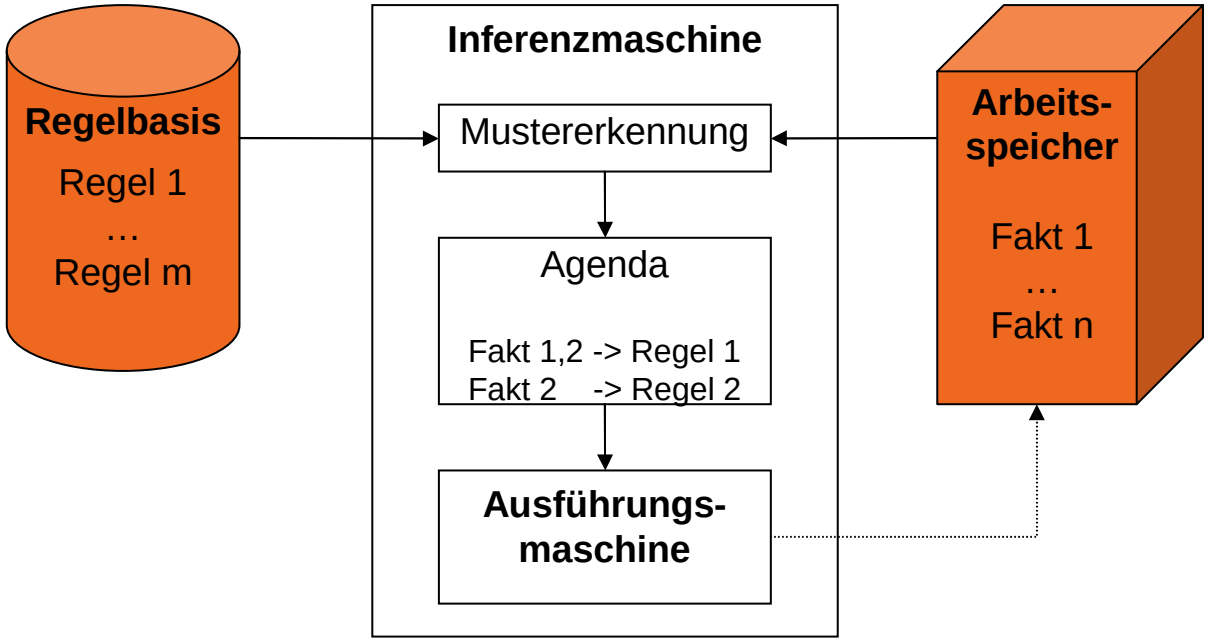
- Eine Regel beschreibt das „was getan wird“ nicht „wie etwas getan wird“
- Regeln sind unabhängig voneinander
- Regeln enthalten keinen Ablaufreihenfolge
- Nicht objektorientiert

Beispiel für eine Regel bei „eBay“

- „Wenn das Mitglied über 10 positive Bewertungspunkte besitzt, erhält es einen gelben Stern
- Regel

```
rule “Yellow star”
  when
    $u : User ( positiveFeedbacks > 10 )
  then
    $u.setUserLevel ( “yellow_star” );
end
```

Rule Engine - Aufbau



Rule Engines arbeiten in Zyklen

Rule Engines – Ein Algorithmus

- Verwenden Algorithmen -wie den Rete-Algorithmus- um Regeln durch eine Vorkompilierung (Optimierung) zur Laufzeit schneller prüfen zu können
- Rete-Algorithmus (Charles Forgy):
  - Arbeitet als Blackbox
  - Umformung der Regeln in ein Bedingungsnetzwerk
  - Bewahrung des Zustandes über mehrere Zyklen hinweg

Agenda

- Rule Engine
- Drools
- API – Level – Integration
- Integrationsszenarien
- Zusammenfassung

Drools

- Aktuelle Version 4.0.4
- Vollwertige Rete Implementierung
- 2006 in JBoss Familie aufgenommen
- 100% Java
- Standalone Produkt
- Arbeitet mit POJOs

Drools Features

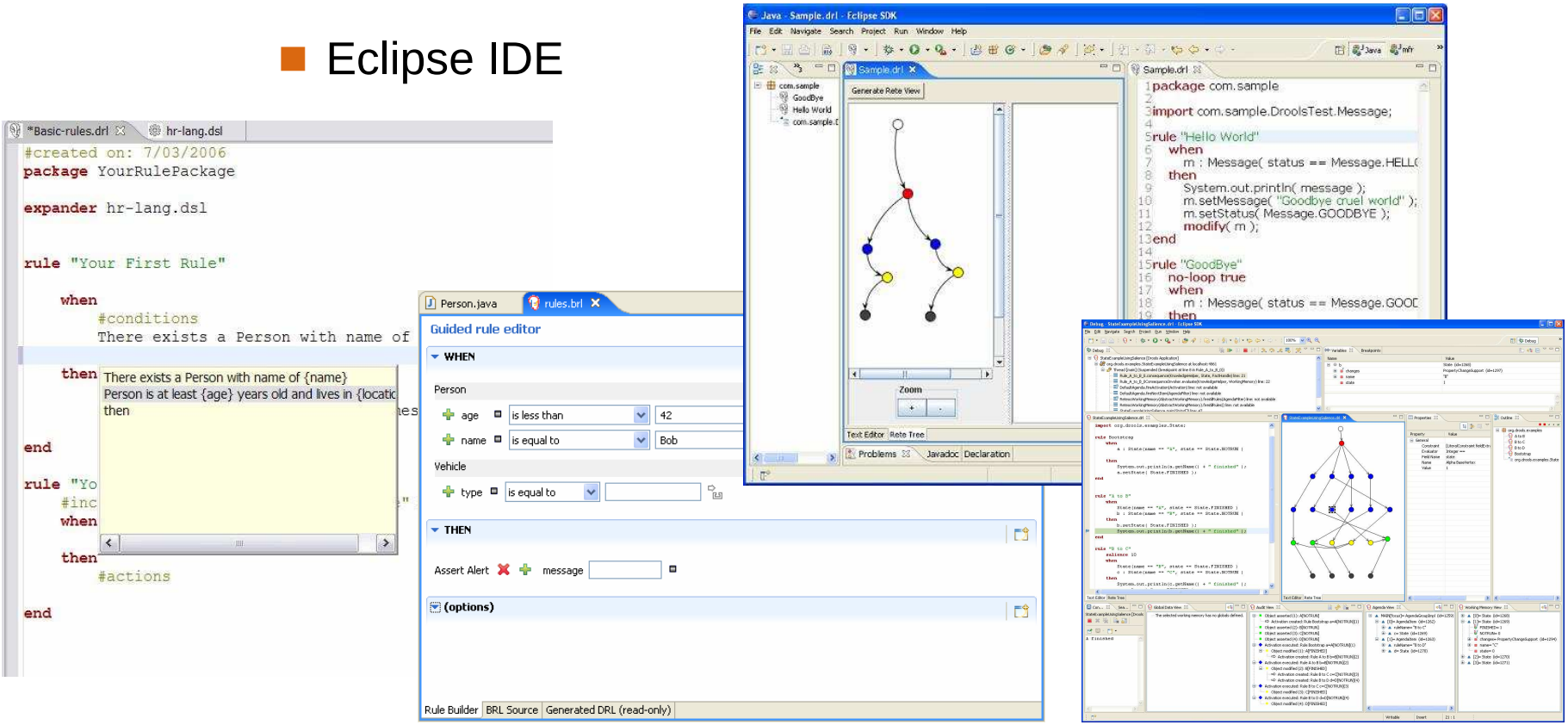
- Inferenzmaschine
- Eigene Regelsprache (DRL)
- Templates für natürlichsprachlichere Regeln (DSL)
- Decision Tables (XLS)

Drools Features

- BRMS-Server (Web 2.0 ;)
- Rule Flow
- Eclipse IDE
- Drools Solver
- JSR 94-Implementierung

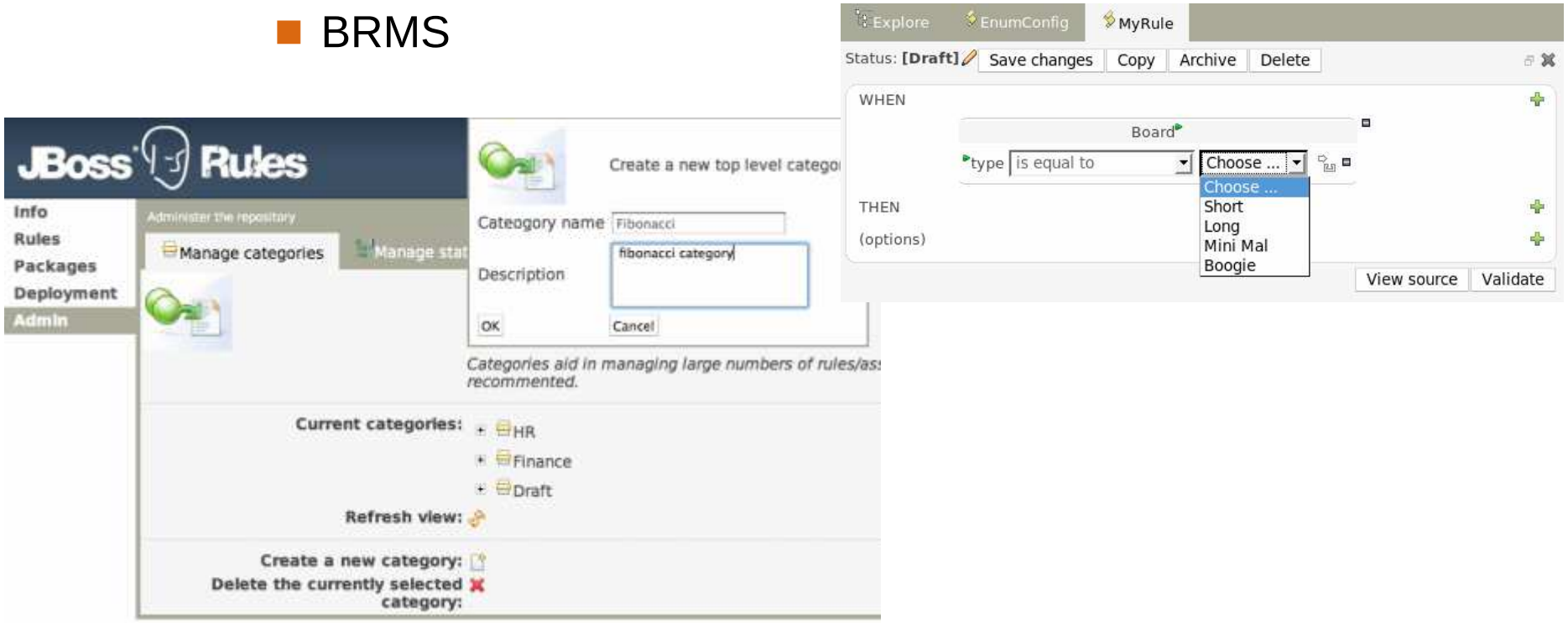
Drools Tooling

- Eclipse IDE



Drools Tooling

- BRMS



Agenda

- Rule Engine
- JBoss Drools
- API – Level – Integration
- Integrationsszenarien
- Zusammenfassung

Java Rule Engine API (JSR 94)

- Standardisiert den Zugriff auf Rule Engines
- Reihe von Interfaces für die Verwaltung von Regelsätzen und deren Nutzung zur Laufzeit zur Verfügung
- Vorteile
  - Entwickler müssen sich nicht mit dem nativen Code einer Rule Engine auseinander setzen
  - Soll den Austausch der Rule Engine erleichtern

Java Rule Engine API (JSR 94)

- Allgemeine Nachteile
  - Enthält **KEINEN** Standard für eine Regelsprache
    → Wechsel der Rule Engine erfordert Portierung sämtlicher Regeln
  - Der Aufwand steckt in den Regeln nicht im Zugriffscode für die Rule Engine
  - JSR 94 – Code ist aufgebläht und nicht so verständlich wie z.B. die native Drools API

Java Rule Engine API (JSR 94)

- Drools-spezifische Nachteile
  - Features nur gegen Abhängigkeit
    - Möglichkeit zur Nutzung von Globals und DSL über Property-Parameter
    - Verlust der eigentlich gewollten Unabhängigkeit!
  - Keine Unterstützung von Rule Flows
  - Keine Möglichkeit zur Nutzung von Queries
  - Ab Drools 5.0: Kein Complex Event Processing

Alternative: Rule Engine Wrapper

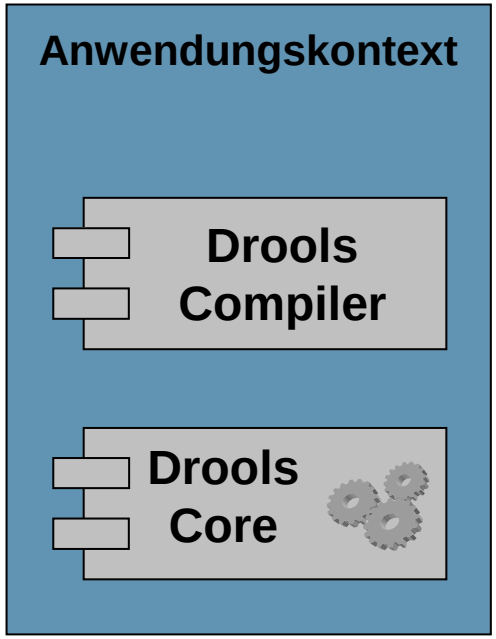
- Native Drools API von der restlichen Architektur durch eigene API kapseln
- Nur für die Verwendung der Rule Engine notwendige Funktionalitäten anbieten
- Rule Engine als „Decision Service“ in Architektur anbieten

Agenda

- Rule Engine
- JBoss Drools
- API – Level – Integration
- Integrationsszenarien
- Zusammenfassung

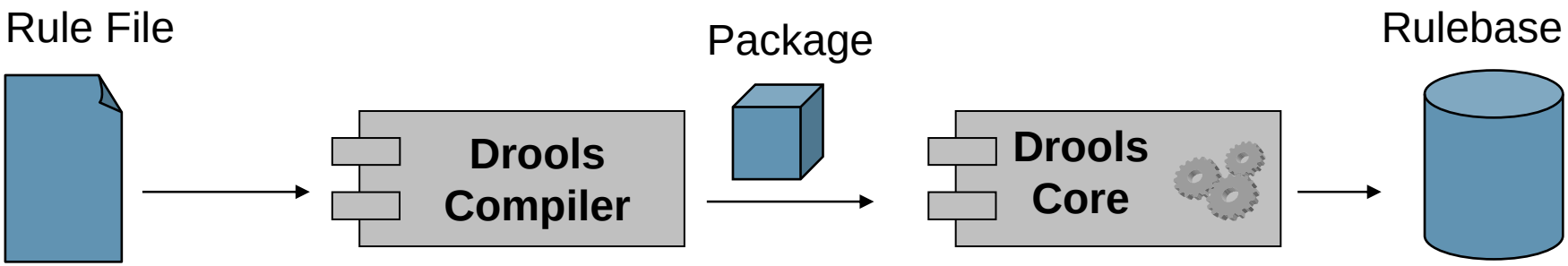
Integrationsszenarien

- Regelkompilierung und Regelausführung im Anwendungskontext
  - Einfachste Umsetzung (+)
  - Zeitfresser Regelkompilierung (-)



Integrationsszenarien

- Trenne Regelkompilierung und Regelausführung!

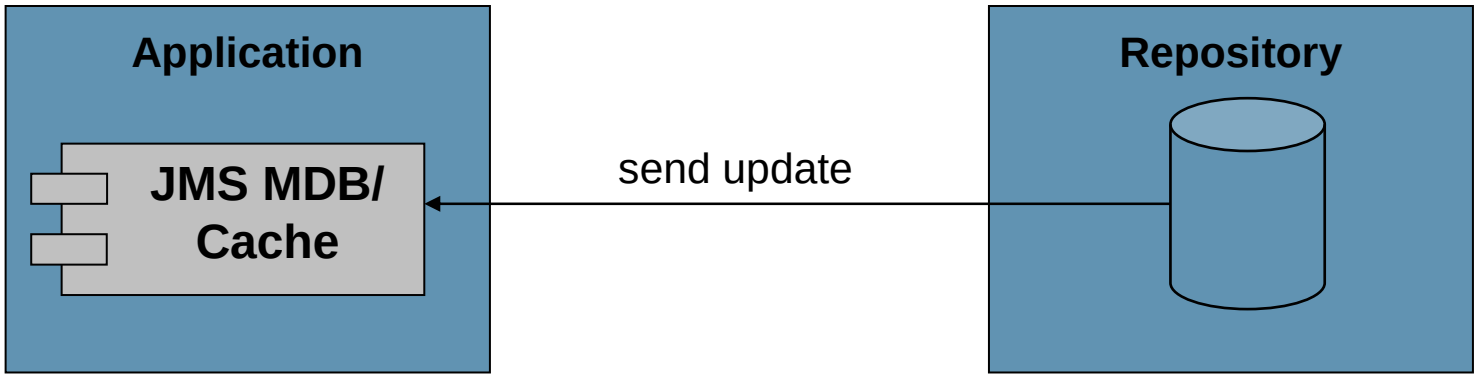


Integrationsszenarien

- Pull-Style: Anwendung holt sich Package/Regeln ab
  - Timer-Komponente (z.B. Quartz)
  - Drools Rule Agent

Integrationsszenarien

- Push-Style: Anwendung erhält Package/Regeln zugeschickt
  - Transport via JMS



Integrationsszenarien

- J2EE-Kontext
  - Package, Rulebase und Rule Agent können über den JNDI (Java Naming and Directory Interface) verfügbar gemacht werden
  - JMX (Java Management Extensions) erlaubt eine einfache Verwaltung der über JNDI angebotenen Ressourcen

Integrationsszenarien

- Framework Integration
  - Entwicklungsplattformen wie JBoss Seam integrieren Drools
  - Spring Integration ist ebenfalls möglich
- Integrationsbeispiele für Drools
  - Apache Servicemix (Enterprise Service Bus)
  - Mule ESB
  - JBoss ESB
  - JBoss jBPM
  - Uvm.

Integrationsszenarien - Tipps

- StatefulSession ohne Referenz in RuleBase
- StatelessSession bei Decision Services verwenden
  - Sequential-Mode steigert evtl. Performance
- Nicht vergessen equals() und hashCode() zu überschreiben!

Integrationsfrage
Ich könnte eine Rule Engine nutzen,

- wenn ich ein komplexes Problem lösen muss
- wenn ich für das Problem Regeln formulieren kann und

  jede Regel >3 Bedingungen hat (Heuristik)
- wenn sich die Regeln wahrscheinlich mit der Zeit verändern
- wenn die Wartung der Regeln garantiert ist
- wenn Performance nicht alles ist (dies bedeutet nicht, dass Rule Engines langsam sind!)
- wenn ich (Geschäfts-)Logik von der Anwendung trennen muss

http://herzberg.ca.sandia.gov/guidelines.shtml

Achtung!

- Zur Verwendung einer Rule Engine wird benötigt
  - Know-how für das verwendete Rule-Engine-Produkt
  - Ein Verständnis für das zu lösende Problem
- Rule Engine erhöhen die Komplexität der Architektur, senken jedoch die Komplexität einzelner Komponenten
- Rule Engines / BRMS brauchen Organisation
  - Wer erfasst/testet/pflegt/deployt die Regeln?
  - Wer wartet die Engine?

Hilfe!

- Hilfe bieten
  - die Dokumentation
  - die mitgelieferten Beispiele
  - die Community via Mailingliste
  - das Internet
  - die Community via IRC
  - die Unit-Tests aus dem Projekt
  - der Quellcode an sich
  - JBoss (offizieller Support)
  - unabhängige IT-Beratungs- und –Servicehäuser

Agenda

- Rule Engine
- JBoss Drools
- API – Level – Integration
- Integrationsszenarien
- Zusammenfassung

Zusammenfassung

- Drools
  - ist die Open Source Rule Engine
  - bringt reichlich Werkzeug mit
  - bietet viele Integrationsmöglichkeiten
- In meinem nächsten Projekt setze ich auf Drools

Referenzen

- http://labs.jboss.com/drools/
- http://rbs.gernotstarke.de/
- http://blog.athico.com/
- http://herzberg.ca.sandia.gov/guidelines.shtml

EOF

```
rule „if there are any questions“
    when
        $q : Question ( )
        $m : Michael ( )
    then
        $m.tryAnswering( $q );
End
```

Danke für die Aufmerksamkeit! Happy Drooling!

Michael Frandsen
syngenio AG
michael.frandsen@syngenio.de
http://www.syngenio.de